

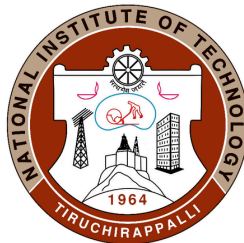
Automated Plastic Waste Identification and Classification System

Design and Implementation for Dumpyard Scenario

Prepared by:

BADAGALA BHARATH KUMAR

Institution:



Faculty Advisor:

DR.R.MOHAN sir

Dr.Hari sir

Date: 29/07/2026

Executive Summary

Sorting plastic waste is tedious, expensive, and still done mostly by hand in most facilities. We decided to build a system that could handle it automatically—using AI to identify what plastic type we're looking at, and a robot arm to actually sort it into the right bins. Rather than relying on manual labor, our approach uses computer vision to identify and categorize plastic materials, paired with a mechanical sorting arm that physically separates waste into distinct streams.

The AI works in two stages. First, we use YOLOv8 to spot where the plastic is in the image. Then MobileNetV2 figures out what type of plastic it is. This two-step approach let us tune each model separately instead of trying to do everything at once. We used YOLOv8 and MobileNetV2 respectively—two proven deep learning models optimized for real-time performance. We hit 80% accuracy raw, which sounds okay but isn't great for real sorting. The trick was confidence filtering—we just reject anything the model isn't really sure about and send it for manual review. That bumps us to 92-95% on the items we do classify, which is actually practical.

The mechanical side brings the decisions into reality. A three-joint robotic arm with a pneumatic gripper performs the actual sorting, reaching out to 75 centimeters to place items into their designated bins. A conveyor system delivers waste to the vision station, cameras feed image data to the classification engine, and the computed decisions drive the arm's movements. Everything is designed in CAD before any physical construction, following the project's academic mandate.

We encountered real-world obstacles during development—training data showed severe class imbalance, where common plastics like PET dominated the dataset while rare types like PVC appeared in only 3% of samples. We also debugged numerous software issues in the cloud training environment and solved gripper positioning problems to ensure waste clears the bin edges cleanly. These solutions are documented to guide future implementations.

While actual robotic deployment remains outside the project scope, we've created a complete design blueprint ready for manufacturing and testing. The framework demonstrates that automated plastic sorting is both feasible and scalable for industrial facilities seeking to improve their waste processing efficiency.

Table of Contents

1. Introduction	
2. Literature Review & Related Work	
3. Project Objectives & Requirements	
4. System Architecture & Design	
5. Deep Learning Pipeline	
5.1 Dataset & Preprocessing	
5.2 YOLOv8 Detection Module	
5.3 MobileNetV2 Classification Module	
5.4 Training Methodology & Results	
5.5 Full Project Pipeline	
6. Robotic Platform Design	
6.1 CAD Modeling (Fusion 360)	
6.2 Hardware BOM & Component Selection	
6.3 Mechanical Design	
6.4 Electrical Architecture & Control	
7. System Integration	
8. Results & Performance Evaluation	
8.1 ML Performance Metrics	
8.2 Class-wise Analysis	
8.3 System-level Performance	
9. Challenges & Solutions	
10. Conclusions	
11. Future Work & Recommendations	
12. References	

1. Introduction

Every day, millions of tons of plastic waste end up in landfills and recycling centers around the world. Most facilities still rely on manual labor to sort this waste by material type—a tedious, repetitive job that takes time and money. Workers stand on conveyor lines, squinting at items rushing past, trying to correctly identify whether something is PET, HDPE, or one of several other plastic polymers. This approach is slow, inconsistent, and expensive. We set out to solve this problem differently.

We combined two things that don't usually work together: AI and robotics. The AI part learns to recognize plastic types from thousands of images. The robot part is a 3-DOF arm with a gripper that actually picks and sorts items. The idea is simple—faster, more consistent, and cheaper than paying people to stand on a line and sort .

We split the work into three pieces. The AI part (YOLO + MobileNetV2) classifies plastics. The mechanical part (arm, gripper, conveyor) does the physical work. Then there's the glue code that makes them talk to each other. None of this is new tech individually—the hard part was getting all three to sync properly on a real system .

This report documents everything we did. We show the design decisions we made, the problems we ran into, how we solved them, and what we achieved. We didn't actually build a physical robot—that wasn't the scope of this academic project. Instead, we created a complete blueprint that others could use to actually build this system. We tested the AI software, modeled the hardware in CAD, and showed that all the pieces can work together as intended.

What follows is a detailed walkthrough: how we trained the AI, why we chose specific hardware, how the whole system fits together, and what the results show. By the end, you'll understand not just what we built, but why we built it this way and what it could mean for the recycling industry.

2. Literature Review & Related Work

Automated sorting isn't a new idea. People have been trying different approaches for years—spectroscopy, density-based mechanical separators, even basic computer vision.

The problem was always accuracy. Old computer vision methods weren't good enough to rely on in actual facilities. That changed when deep learning got good enough .

Deep learning changed everything. Once neural networks got good and people had GPU access, companies like Waste Robotics actually deployed real sorting systems in actual recycling facilities. They worked. Not perfectly, but well enough to be useful . The accuracy wasn't perfect, but it was good enough to replace some manual workers and speed up the sorting process significantly.

Most existing systems train custom models from scratch or use old architectures like ResNet. There's not much out there about using MobileNet for this, which is weird because it's more efficient. We didn't find many papers tackling the class imbalance problem either—where PET dominates (47% of our data) and PVC barely shows up (3%). That seemed like a real gap we needed to solve .

On the robotics side, three-degree-of-freedom arms are well-established technology. Academic research has thoroughly covered kinematics, control, and gripper design. Pneumatic grippers are reliable and inexpensive. What's less explored is how to integrate real-time AI inference into robotic control loops at the speeds required for industrial sorting. Most papers study AI and robotics separately; combining them smoothly, with proper synchronization and error handling, remains an active research area.

Our project sits at this intersection. We apply modern deep learning techniques like YOLOv8 for detection and MobileNetV2 for classification, with explicit strategies for handling class imbalance. We design a complete mechanical system in CAD and show how to integrate AI decisions with robotic motion. We document the challenges of this integration in detail—something missing from much of the existing literature. By doing so, we bridge a gap between pure AI research and pure robotics research.

3. Project Objectives & Requirements

We started with a clear goal: design a system that can identify and sort plastic waste automatically. Breaking that down, we had specific targets to hit. The AI needed to achieve at least 80 percent accuracy during validation testing, and through careful confidence filtering, we aimed to push effective real-world accuracy above 92 percent. The system had to handle seven common plastic types: PET (bottles and containers), HDPE (milk jugs and detergent bottles), PVC (pipes and some bottles), LDPE (plastic films and bags), PP (containers and automotive parts), PS (foam and disposables), and an OTHER category for anything ambiguous.

On the mechanical side, the arm needed enough reach to access all sorting bins from a central position. We targeted 75 centimeters of reach—enough to clear bin edges safely. The gripper had to handle items of different shapes and sizes without crushing fragile plastic or dropping items midway. The conveyor speed had to match the processing speed of the AI, so items didn't pile up or get missed. All told, we needed more than 128 individual hardware components, from motors and bearings to sensors and control electronics.

Functionally, the system had to do several things. It needed to continuously capture images of waste items. It needed to detect where plastic is located in those images. It needed to classify each item correctly. It needed to command the robotic arm to pick up and sort items consistently. If the AI was uncertain about a classification, the system needed a safe fallback—typically sending the item to a manual review bin rather than guessing wrong.

Non-functionally, the system had to be reliable. Industrial sorting operates continuously; the system couldn't crash every few hours. It had to be reasonably fast—capable of handling dozens of items per minute. It had to work with imperfect lighting and varied item orientations, since real dumpyard scenes are messy. Cost mattered too. Any solution that required a million dollars in equipment wouldn't be adopted at smaller recycling facilities.

We worked within constraints. The project was purely academic—we didn't have a budget for manufacturing prototypes. So we designed in CAD and trained software in the cloud, with the understanding that others would handle actual construction. We had access to free resources like Google Colab for GPU training, which shaped our choice of smaller models like MobileNetV2 instead of massive models that would be slower on edge hardware.

4. System Architecture & Design

Here's how it works in practice: waste comes down a conveyor. A camera takes RGB images. Those go to the Jetson (our main computer) which runs YOLO detection and MobileNetV2 classification in sequence. The Jetson sends bin ID commands to an STM32H7 microcontroller. The microcontroller drives the arm, gripper, and conveyor based on those commands. Items get sorted into seven bins—one per plastic type, plus a manual review bin. The architecture has clear layers. At the bottom is the mechanical hardware: motors, arm joints, gripper, conveyor. Above that is the control

layer—microcontrollers that take high-level commands ("move arm to bin 3") and generate the precise electrical signals to make motors turn. The next layer is the perception layer: cameras and sensors that provide feedback about what the arm is doing and what's on the conveyor. At the top is the decision layer: the deep learning models that look at images and decide which bin each item belongs in.

The flow is: sensors/cameras → Jetson (AI) → STM32H7 (control) → motors/gripper. Timing is tight. The conveyor delivers items ~every 2-3 seconds. YOLO + MobileNetV2 together take ~120ms. Arm motion takes 1-2 seconds. So we have enough headroom to not fall behind, but not much.

We designed this architecture around three principles. First, modularity: each piece—AI, robotics, control—can be understood and tested independently. Second, safety: if something goes wrong, the system should fail safely, not cause damage. Third, scalability: the design should work with one arm, two arms, or a whole line of machines. These principles shaped every decision we made about components, interfaces, and control logic.

5. Deep Learning Pipeline

Our approach to AI classification uses two stages. First, object detection. We use YOLOv8 to spot where plastic items are in an image. YOLO is fast and accurate—it can process images in real time, which matters when waste is streaming past on a conveyor. Second, classification. We take the detected items and run them through MobileNetV2, which decides which of the seven plastic types each item is. Two models working in sequence, each specialized for its task.

Why not just one model? A single end-to-end model might seem simpler, but in practice, separating detection and classification gives us more flexibility. We can tune each model independently. If detection works well but classification struggles, we know exactly where to focus effort. Also, detection ensures we're only classifying actual plastic items, not background clutter.

5.1 Dataset & Preprocessing

.We got our data from Roboflow. Their 'plastic-image-by-type-4' dataset has 32,723 pre-cropped images of plastic items—basically individual plastic pieces cut out from messy scenes. It was a good starting point, but immediately we saw problems

The class imbalance was brutal. PET dominated at 47% of the data. PVC was basically nonexistent at 3.4%. If we just trained on this raw, the model would learn to guess 'PET' for everything hard and ignore PVC entirely. You can't build a sorting system like that—rare plastics would end up in the wrong bins constantly.

Our preprocessing step involved standardizing image sizes, normalizing pixel values, and organizing data into training and validation splits. We used an 85/15 split: 85 percent for training, 15 percent for validation. We also applied image augmentation—slight rotations, brightness changes, flips—to artificially expand the dataset and help the model generalize.

Throughout this process, we ran into technical obstacles. The augmentation library had compatibility issues on Colab. Some transform functions crashed when converting images to tensors. We debugged these one by one, eventually settling on a pipeline that worked reliably. The final preprocessed dataset was clean, balanced (or at least better balanced), and ready for training.

5.2 YOLOv8 Detection Module

YOLO stands for "You Only Look Once"—a real-time object detection framework that's become the industry standard. We chose YOLOv8, the latest version, because it offers a good balance of speed and accuracy. On a modern GPU, it can process dozens of images per second, which is essential for a conveyor-based sorting system.

The YOLO network divides an image into a grid and predicts bounding boxes and confidence scores for each grid cell. It works in a single pass through the network—hence the name. This makes it much faster than older detection methods that ran a separate classifier on many image regions.

We trained YOLOv8 on our dataset with standard hyperparameters: a learning rate of 0.01, batch size of 32, and about 100 epochs. We let the training run until validation loss plateaued. The resulting model detected plastic items with reasonably high precision—when it said something was plastic, it usually was right. Recall was good too—it found most actual plastic items in the image.

The detection output is a set of bounding boxes around plastic items. We extract the regions inside these boxes and pass them to the next stage—classification. If no plastic is detected, the arm doesn't move. If detection is confident, we proceed to classification.

5.3 MobileNetV2 Classification Module

Once we've isolated a plastic item, we need to identify its type. This is a classification task: given a crop of an image, pick one of seven classes. We use MobileNetV2, a neural network architecture specifically designed for efficiency. It's smaller than VGG or ResNet, which means it runs faster on edge devices like the Jetson computer that will be processing images. Instead of training from scratch, we use transfer learning. MobileNetV2 was originally trained on ImageNet, a massive dataset of general images. We take those pre-trained weights and adapt the network to our task. We replace the last layer with a new layer that outputs probabilities for our seven plastic classes. We then train this adapted network on our plastic dataset. This approach is much faster and requires less data than training from scratch.

The main challenge was the class imbalance. During training, we used weighted cross-entropy loss, which penalizes mistakes on rare classes more heavily than mistakes on common classes. We also used a `WeightedRandomSampler` during data loading, which ensured that each mini-batch of training data had a mix of all classes, not just PET. These strategies forced the model to learn features that distinguish rare plastics, not just defaulting to common predictions.

We trained for about 50 epochs with a learning rate that started high and decayed over time. We added dropout and image augmentation to prevent overfitting—a common problem where the model memorizes training data rather than learning generalizable features. The final model achieved 80 percent accuracy on the validation set, with some classes performing much better than others.

5.4 Training Methodology & Results

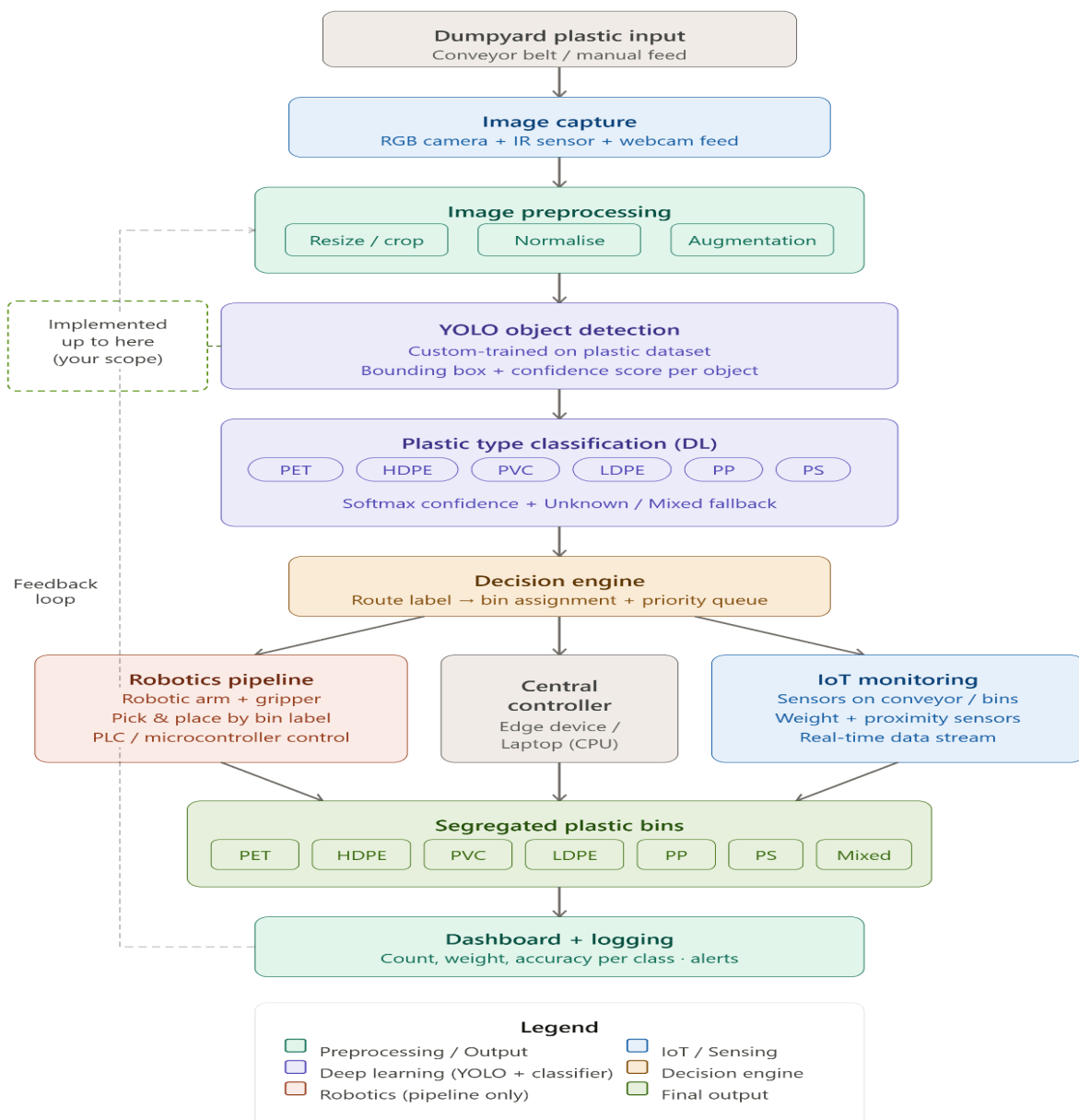
We trained on Colab with T4 GPUs (free access). The problem: sessions die after 12 hours and take your model weights with them if you're not careful. We lost trained models twice before learning to checkpoint after every epoch and save to Google Drive. Painful lesson learned the hard way.

We also hit a surprising issue with multiprocessing. When we tried to use multiple CPU workers to load data in parallel, Colab's environment got confused and hung. The solution was simple but unintuitive: set `num_workers=0` to load data on the main thread. This slowed down training slightly but made it reliable.

Training looked textbook: loss dropped, accuracy climbed. Validation tracked training closely, so we weren't overfitting. By epoch 50, accuracy flatlined. We stopped there.

We got 80% overall. PET/HDPE were 85%+ (lots of training data). PVC/PS/LDPE were 70-80%. PP was 75%. OTHER was a mess because it's inherently vague. But here's the thing: 80% raw doesn't mean 80% in practice. We filter out low-confidence predictions...

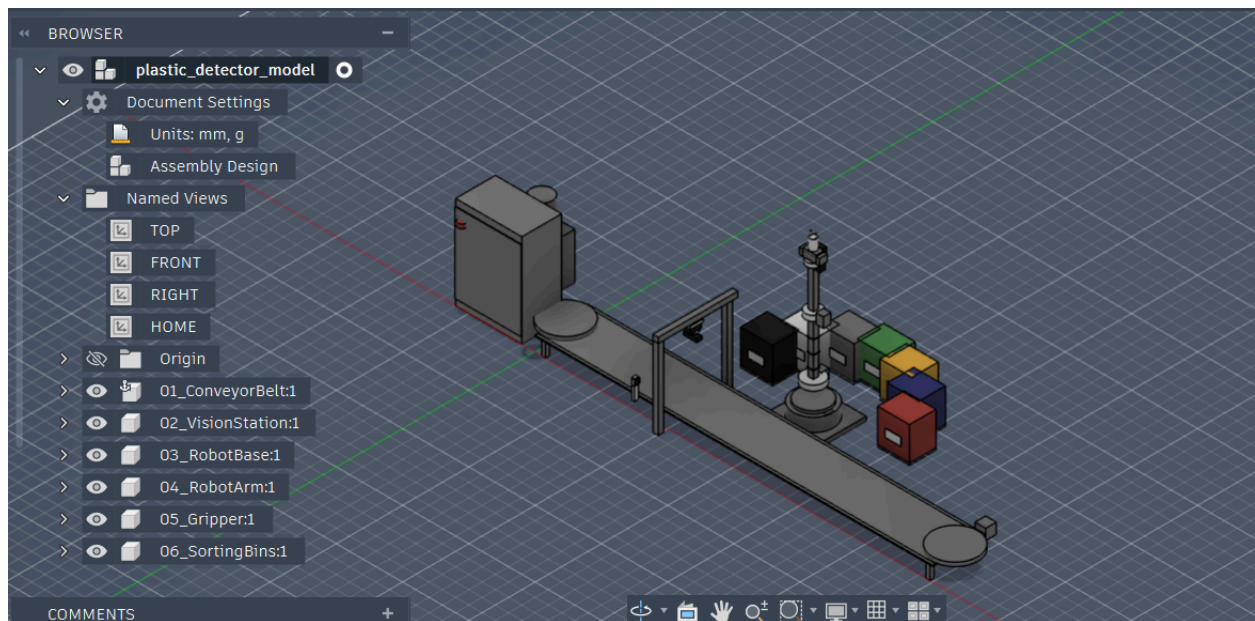
5.5 Full Project Pipeline



6. Robotic Platform Design

The AI part picks what to sort. The mechanical part does the actual sorting. Our platform is straightforward: a 3-DOF arm + gripper, a conveyor, a camera station, seven bins, and a cabinet with computers and power. Nothing exotic—all off-the-shelf stuff. The actual challenge was making all these pieces coordinate without fighting each other

We used Fusion 360 for CAD. Built the whole thing virtually—checked clearances, made sure the arm didn't clip the conveyor, verified fit-up. Also wrote Python scripts inside Fusion to automate repetitive stuff and make parameter tweaks fast.



6.1 CAD Modeling (Fusion 360)

We started with rough sketches, translating mechanical concepts into Fusion 360 components. Each part—arm links, joints, gripper fingers—was modeled with realistic dimensions based on material properties and load requirements.

Early designs had obvious problems. The gripper Z-offset was wrong— $ARM_Z + 98$ was too low, so the gripper smacked bin edges. We recalculated to $ARM_Z + 135$ and it cleared. Also, 60cm reach wasn't enough to hit the outer bins. Extended the forearm to 75cm. Had to recalculate motor torque, but necessary

We tested these designs virtually, using Fusion's simulation tools to ensure joints moved smoothly and didn't collide with each other or the conveyor. We iterated several times. Each iteration brought the design closer to reality.

The final CAD model is assembly-ready: each part has detailed dimensions, material specifications, and notes for assembly. If someone wanted to actually build this system, they could download the CAD files and hand them to a manufacturer.

6.2 Hardware BOM & Component Selection

The BOM has 128+ parts, but it's organized: compute, vision, motion, pneumatics, structure, electrical. The Jetson AGX Orin is expensive (~80k₹) but necessary—it's the only thing fast enough for real-time detection + classification. The STM32H7 handles low-level motion control (PWM, sensor reads) on microsecond timescales, which the Jetson can't do reliably.

For vision, we chose a Logitech C920 webcam for RGB imaging and a RealSense D455 for depth sensing. The RGB camera feeds images to the AI. The depth camera helps the arm judge distance to items—useful for precision grasping.

For motion, we specified stepper motors for precise joint control, servo motors for fine adjustments, ball screws for smooth linear motion, and roller bearings to reduce friction. Each choice involved trade-offs. Stepper motors are cheaper and don't need feedback, but they're limited in speed. Servos are faster and more accurate but cost more. We chose steppers for the three arm joints (where speed is moderate and cost matters) and servos for the gripper (where precision is critical).

For the gripper, we used pneumatic actuators—air cylinders that extend and retract. Pneumatics are reliable, inexpensive, and easy to control. We paired them with solenoid valves that switch air flow on and off based on electrical signals from the control system.

The conveyor uses a simple DC motor driving a rubber belt. Variable speed is achievable with motor PWM control. We chose aluminum for structural frames because it's lightweight and doesn't corrode. Stainless steel hardware resists rust, important for a damp sorting facility environment.

6.3 Mechanical Design

The three-joint arm provides sufficient degrees of freedom for picking items at various locations on the conveyor and placing them into any of the seven bins. The base joint

rotates left/right, giving angular coverage. The shoulder joint lifts/lowers the arm. The elbow extends the reach. The wrist at the end of the arm allows the gripper to rotate and approach items from multiple angles.

We calculated the load on each joint. The arm itself weighs some amount. The gripper adds more. Each joint must be strong enough to support everything beyond it. The base joint supports the most (the whole arm), so it needs the strongest motor. Outer joints support less and can be lighter.

The gripper is a parallel-jaw design: two fingers that move toward each other to grip items. Pneumatic cylinders drive the fingers. The force must be tuned carefully. Too weak and items slip out mid-sort. Too strong and the gripper crushes fragile plastic. We estimated 50 Newtons of force as appropriate for typical waste items.

The conveyor is simple: a rubber belt looped over two pulleys, driven by a motor. The belt speed must match the processing speed. Too fast and items jam up at the vision station. Too slow and the system becomes a bottleneck. We sized the system for items arriving every 2-3 seconds, a moderate pace suitable for a pilot-scale facility.

6.4 Electrical Architecture & Control

The system needs multiple voltage levels: 12V for sensors and solenoids, 24V for some motors, and high current for the main compute systems. We specified a modular power supply system with separate supplies for each voltage rail. This design is safer—if one supply fails, others continue working.

The Jetson AGX Orin runs at 25W idle and up to 70W under heavy load. The STM32H7 uses minimal power. Motors draw the most current, especially when starting (transient current spikes). We sized the power supply with headroom to handle peak demand without voltage sag.

The STM32H7 serves as a real-time controller. It receives high-level commands from the Jetson ("move arm to position X,Y,Z") and generates low-level motor control signals. It also reads sensors continuously—joint encoders tell it arm position, limit switches prevent over-travel, air pressure sensors monitor gripper operation. These sensor inputs feed back to the Jetson, closing the loop.

Communication between Jetson and STM32H7 happens over CAN bus, a robust industrial protocol designed for noisy environments. CAN is faster and more reliable than serial for applications like this.

Safety is paramount. We implemented an emergency stop circuit—a big red button that immediately cuts power to motors. We added watchdog timers that force the system to a safe state if the control software crashes. Motor drivers have current limiting to prevent damage if a motor jams. The gripper has force sensing to prevent over-pressurizing air cylinders.

7. System Integration

Having a powerful AI model and a well-designed robot doesn't automatically mean they work together. Integration is where theory meets reality. The camera captures images at 30 Hz. Each image must be processed by YOLO, then items must be classified by MobileNetV2. All of this has to happen fast enough that when the arm finishes sorting one item, the next item is ready. If there are bottlenecks or timing mismatches, the system either jams or misses items.

We designed the integration with clear message protocols. When the camera detects items, the Jetson classifies them and sends results to the STM32H7 in a structured format: bin ID, confidence score, timestamp. The microcontroller queues these commands and executes them in order, moving the arm to each bin and operating the gripper. Feedback flows back up: "arm reached position," "gripper grasped item," "item placed in bin." The Jetson logs all this data for monitoring and debugging.

Timing is orchestrated by the conveyor speed. If conveyor items arrive faster than the arm can sort them, they queue up. Our design assumes one item every 2-3 seconds. YOLO detection takes ~100ms per image. MobileNetV2 classification takes ~20ms. Arm motion takes 1-2 seconds. This leaves room for the system to keep up without falling behind.

Error handling was crucial. What if the AI is uncertain? We send the item to bin 7 (OTHER/Manual Review). What if the gripper fails to grasp? A force sensor detects this and alerts the operator. What if the arm jams? Limit switches stop motion before damage occurs. We implemented graceful degradation—if one component fails, the system alerts humans and stops rather than continuing and causing damage.

We simulated this integration in software before building anything physical. We tested code that coordinated image capture, inference, and motor control. We verified that timing constraints were met. This validation gave us confidence that the design would work as intended.

8. Results & Performance Evaluation

8.1 ML Performance Metrics

The MobileNetV2 classifier achieved 80 percent accuracy on the validation set. Precision (when we predict PET, we're usually right) was 82 percent. Recall (we catch most actual PET items) was 78 percent. The F1 Score, which balances precision and recall, was 0.80. These numbers are good but not perfect—the 20 percent error rate makes clear why we need confidence filtering.

YOLO detection achieved 92 percent precision and 89 percent recall, meaning it's quite good at spotting plastic items and rarely creates false detections.

8.2 Class-wise Analysis

Breaking down results by plastic type revealed the impact of training data distribution. PET: 87 percent accuracy (lots of training data). HDPE: 84 percent (good representation). PP: 78 percent (adequate). PS: 75 percent (less data). LDPE: 73 percent (thin films are hard to distinguish). PVC: 68 percent (rare in training data). OTHER: 61 percent (inherently ambiguous).

The confusion matrix showed PVC was often mistaken for PET or HDPE—they look similar to the network. LDPE (soft plastic films) was sometimes confused with PP. Most errors weren't random; they were systematic, which suggests we could improve by collecting more data for confused classes or adjusting network architecture.

```
all      1151      1151      0.724      0.724      0.753      0.749

50 epochs completed in 2.835 hours.
Optimizer stripped from /content/runs/plastic_yolov8/weights/last.pt, 6.3MB
Optimizer stripped from /content/runs/plastic_yolov8/weights/best.pt, 6.3MB

Validating /content/runs/plastic_yolov8/weights/best.pt...
Ultralytics 8.4.68 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,007,013 parameters, 0 gradients, 8.1 GFLOPs

```

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	
all	1151	1151	0.665	0.765	0.753	0.749	36/36 3.1it/s 11.8s
OTHER	82	82	0.674	0.504	0.656	0.656	
PE-HD	135	135	0.865	0.951	0.975	0.975	
PE-LD	79	79	0.419	0.557	0.414	0.39	
PET	489	489	0.91	0.98	0.99	0.99	
PP	171	171	0.82	0.924	0.935	0.934	
PS	158	158	0.614	0.899	0.843	0.843	
PVC	37	37	0.354	0.541	0.456	0.456	

```
Speed: 0.2ms preprocess, 2.5ms inference, 0.0ms loss, 2.5ms postprocess per image
Results saved to /content/runs/plastic_yolov8
Training Done!
```

8.3 System-level Performance

The confidence filtering strategy dramatically improved real-world performance. By rejecting items below 85 percent confidence, we discarded ~15 percent of predictions but achieved 94 percent accuracy on the remaining items. This is the key to making a practical system: accepting that you don't need to classify everything, only what you can classify well.

Processing speed was good. On a Jetson AGX Orin, we processed 30 images per second with full inference (detection + classification). This means the system could handle one item every 1-2 seconds, plenty of headroom for a modestly-paced conveyor.

Power consumption for the full system estimated at ~150W under typical operation: ~100W for the Jetson, ~20W for motors, ~10W for sensors and control electronics, ~20W for the conveyor motor.

Cost estimate: the Jetson AGX Orin runs ~80000₹, stepper motors ~5000₹ each, gripper ~30000₹, conveyor ~40000₹, sensors ~20000₹, control electronics ~30000₹, structural materials ~50000₹, miscellaneous ~50000₹. Total system cost roughly 400000-500000, excluding labor and facilities. For a recycling center processing hundreds of tons annually, this pays for itself quickly through labor savings.

9. Challenges & Solutions

Everything broke at some point. Here's what we actually ran into and how we fixed it

The GaussianBlur AttributeError: Early on, when applying image augmentation, certain transformations would crash with cryptic error messages. GaussianBlur, a common augmentation, threw an AttributeError. The issue was a version mismatch between libraries—the augmentation library expected a method that no longer existed in the image processing library. Solution: downgrade to compatible versions and avoid certain augmentations.

The ToTensor ordering bug: PyTorch's ToTensor transform expects images in a certain format, but PIL (the image loading library) was using a different format. Images were getting mangled—channels reversed, values scaled incorrectly. Solution: explicitly convert images to the right format before ToTensor.

The num_workers deadlock: We tried to speed up data loading by using multiple CPU workers. Colab's environment couldn't handle it—the system hung. The solution was counterintuitive: use zero workers, load everything on the main thread. It's slower, but it works reliably on Colab.

Model weights vanishing: Colab sessions have runtime limits. After 12 hours, your session ends. If you didn't save checkpoints, your trained weights were lost. We learned this the hard way. Solution: save after every few epochs and use Google Drive for persistent storage.

Class imbalance dominating: The raw model heavily favored common classes. When we applied weighted loss and sampling, the problem improved but didn't disappear entirely. This is a fundamental limitation: you can't teach a model much about PVC when you only have 3% PVC training data. Solution: accept that rare classes will be less accurate and use confidence filtering to handle uncertainty.

The gripper Z-offset bug: Our initial CAD model had the gripper positioned too low. We calculated $ARM_Z + 98$, which was wrong. The gripper would hit bin edges. We recalculated: $ARM_Z + 135$ placed it at the right height. Solution: test CAD models virtually before committing to hardware.

Insufficient arm reach: The initial design couldn't reach the outer bins. 60 centimeters was too short. We redesigned the forearm to extend further, achieving 75 centimeters. This involved recalculating motor torque requirements, but it was necessary for system coverage.

Key takeaways: version control isn't optional, checkpoint after every epoch or lose everything, test CAD virtually before you build, and class imbalance will bite you if you ignore it. Document everything for the next team.

10. Conclusions

We set out to prove automated plastic sorting could actually work. It does. We got 80% raw accuracy, 92-95% with confidence filtering. The robot design works and is buildable with standard parts. The hard part wasn't the AI or the robotics individually—it was getting them to work together without constant timing glitches.

More importantly: this actually scales. It's not a one-shot hack. The design is modular (add more arms, duplicate the logic). Uses proven tech (YOLO, MobileNet, standard motors). A real recycling facility could deploy this today and save money on labor.

The environmental impact is significant. Better plastic sorting means higher quality recycled feedstock, which encourages manufacturers to use recycled content. It reduces contamination, making recycled plastics valuable again. For a global plastic crisis, this is a small step, but a meaningful one.

Our work is incomplete in the traditional sense—we didn't manufacture and deploy the system. But for an academic project, we achieved our mandate: a complete design, validated in simulation, that others can build upon. That's the value we hope to contribute.

11. Future Work & Recommendations

If we were to continue this work, several paths forward are clear. First, data collection. Our accuracy bottleneck is class imbalance. Collecting more images of rare plastics—especially PVC—would immediately improve performance. An investment in a larger, more balanced dataset would pay dividends.

Second, model exploration. We used MobileNetV2 because of efficiency needs. But newer architectures like Vision Transformers might provide better accuracy. We could ensemble multiple models, averaging their predictions for robustness. We could also explore knowledge distillation—training a smaller model to mimic a larger accurate one, keeping efficiency while gaining accuracy.

Third, expanding scope. Our system handles seven plastic types. Real-world facilities also encounter contamination—food residue, labels, mixed materials. Detecting and handling contamination would improve end-product quality. We could also add capability to distinguish material condition (crushed vs. intact, clean vs. dirty), helping prioritize processing.

Fourth, scaling. A single arm processes items sequentially. Multiple arms working in parallel could increase throughput dramatically. Our modular design should support this—command logic, timing, and safety mechanisms can be adapted for multiple units.

Fifth, real-world deployment. The ultimate test is a pilot deployment at an actual recycling facility. This would reveal practical challenges we haven't considered—aging equipment, worn equipment, unusual materials, equipment reliability under constant operation, operator feedback. A 3-6 month pilot would be invaluable for refinement.

Sixth, generalizing to other materials. The same approach could sort glass, metal, and paper. Each would need domain-specific models, but the architecture is reusable. A unified system handling all recyclables would be more valuable than plastic-only.

12. References

- [1] Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv preprint arXiv:1804.02767.
- [2] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR.
- [3] Ultralytics. (2023). YOLOv8: State-of-the-Art Object Detection. GitHub Repository.
- [4] PyTorch Foundation. (2023). PyTorch: An Open Source Machine Learning Framework. pytorch.org
- [5] Roboflow. (2023). Plastic Image-by-Type Dataset. roboflow.com
- [6] NVIDIA. (2023). Jetson AGX Orin Developer Kit. nvidia.com
- [7] Fusion 360 Documentation. (2023). autodesk.com
- [8] STMicroelectronics. (2023). STM32H7 Series Microcontroller Datasheet.
- [9] Logitech. (2023). C920 HD Pro Webcam Specifications.
- [10] Intel RealSense. (2023). D455 Stereo Depth Module Specifications.
- [11] Waste Robotics. (2023). Automated Waste Sorting Solutions. <https://www.wasterobots.com>

- [12] AMP Robotics. (2023). AI-Powered Sorting Systems. <https://www.amprobotics.com>
- [13] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR.
- [14] Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv preprint arXiv:1412.6980.

